

Intro to Software Architecture

★ Software Design & Architecture ★



agenda



- Basics of Software Architecture
- C4 model
- Sequence Diagrams and Mermaid Coding



- Next week (Week 9), lab attendance is mandatory to complete lab09.
- Nothing to submit; your grade will largely be based on your level of engagement in the activity.

some admin

software architecture

The fundamental high-level structure of a software system. Defines how system components are structured, how they interact and the guiding characteristics that shape its evolution.

Specifically, it focuses on:

- How the system is divided into modules, services, and layers for the frontend, backend, database, etc.
- How data flows between different components
- What technologies, frameworks or infrastructure are used
- Architectural characteristics that should be addressed (e.g. security, scalability, performance, etc.)

software architecture

Software Design

How individual parts of a system are implemented. Includes detailed decisions about classes, methods, controllers and design patterns.

Software Architecture

Operates at more macro-level, system-wide and involves more structural decisions that take more effort to change. Typically involves multiple components or subsystems.

Example

For communication, “services communicate via RESTful APIs” (architectural) vs. “use the observer pattern to update UI when data changes” (design).

why learn software architecture?



Helps manage complexity especially with large systems, dividing it into manageable, understandable parts.

Software is never static; it continually grows, adapts and changes according to new requirements and the architecture can guide the technical vision and direction of teams in the long-term.





Purpose

- Helps teams visualise and communicate the software architecture of a system clearly at different levels of abstraction.
- Bridges the gap between high-level system overviews and low-level code details, ensuring alignment between stakeholders, developers, and architects.
- Made up of four components, all starting with C!



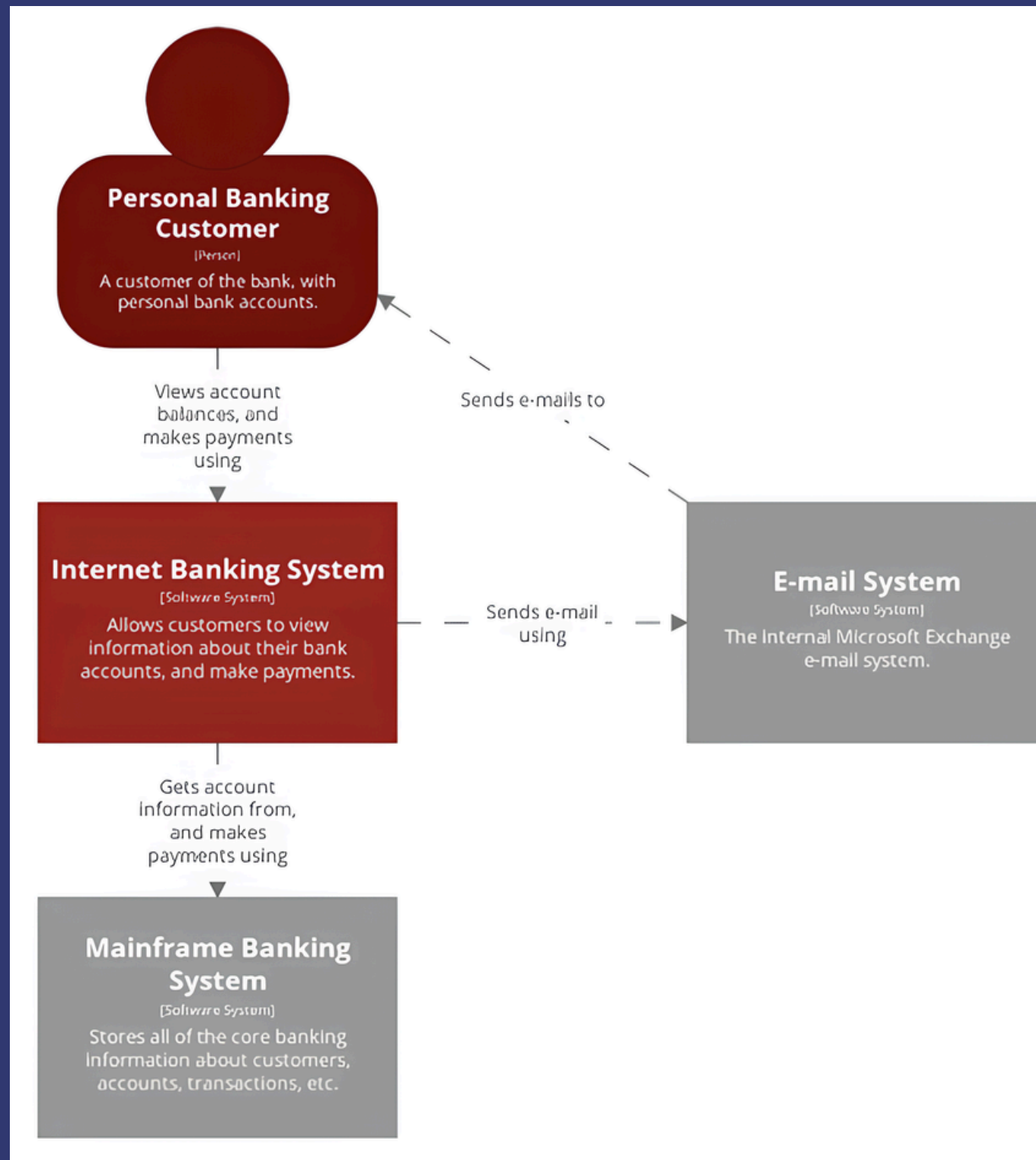
the C4 model

From least to most detailed:

1. **Context Diagram:** Shows the system as a "box" and its interactions with users and external systems.
2. **Container Diagram:** Breaks the system into containers (applications, services, databases) and shows how they interact.
3. **Component Diagram:** Zooms into a specific container to show its internal components and their relationships.
4. **Code (Class) Diagram:** Offers a detailed view of the source code structure (e.g., classes and interfaces) within a component.

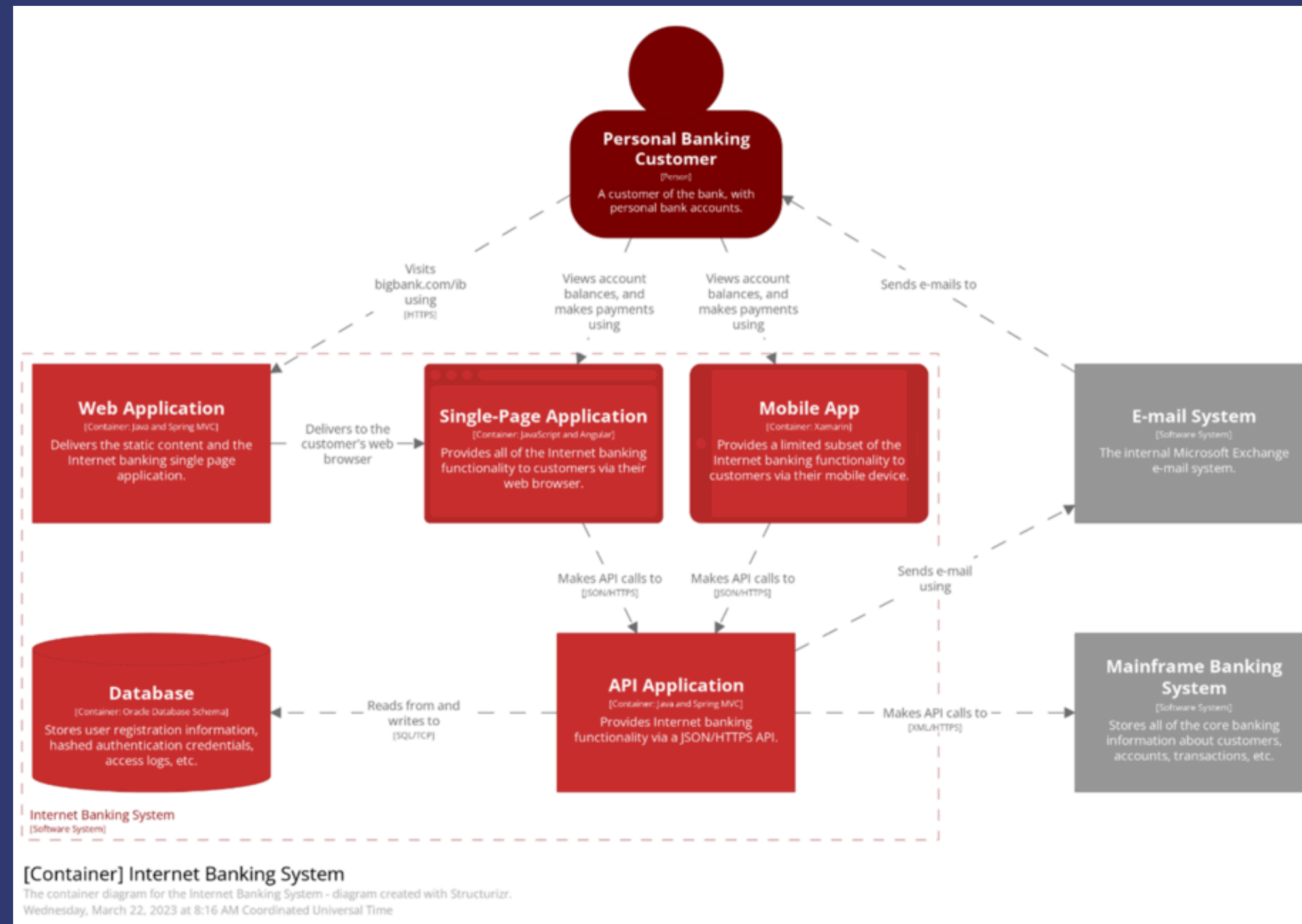


the C4 model



- Intended audience
 - Non-technical stakeholders (business sponsors, product owners, etc.)
 - External users/clients
 - Enterprise architects
 - Anyone else who needs to understand what the system is, what it interacts with, and who uses it.
- Communicates scope and external dependencies of the system in a simple, accessible way.

level one: context diagram



Breaks the system into containers (applications, services, databases) and shows how they interact.

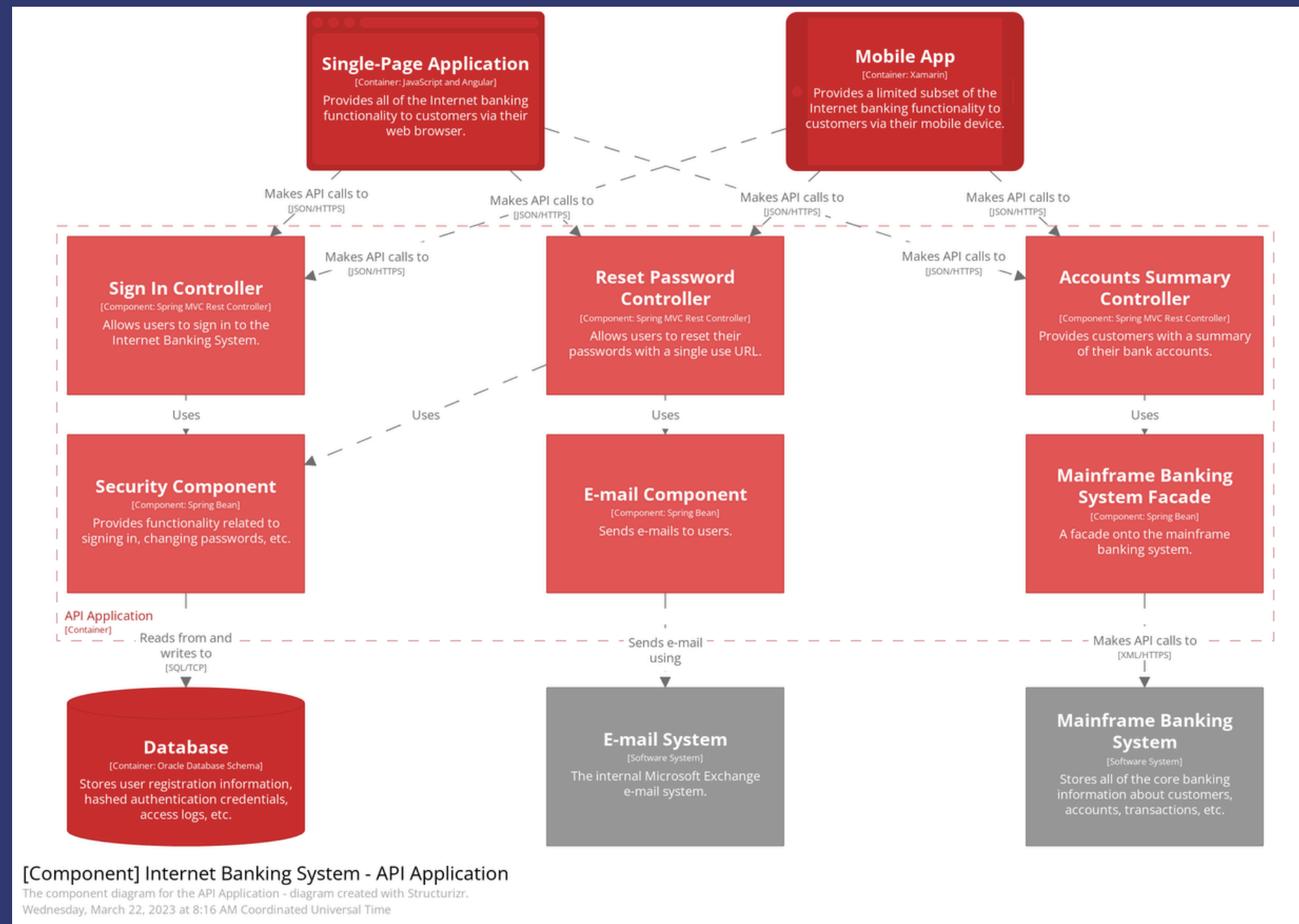
Why separate the web frontend and backend service into different containers?

level two: container diagram



- **Separation of concerns:** UI logic is handled in the frontend; business logic and data processing in the backend.
- **Independent deployment:** You can update the frontend without touching the backend and vice versa.
- **Scalability:** Backend and frontend can be scaled independently depending on load.
- **Technological flexibility:** Different teams can use the most suitable tech stacks for each (e.g. React frontend, Node.js backend)
- **Security:** Backend can be isolated in a secure environment, while frontend runs in browsers.

level two:
container diagram



Zooms into a specific container to show its internal components/controllers and their relationships.

Sounds similar to a container diagram, so what's the difference?

level three: component diagram



Container

- Focuses on the **major deployable units** (web apps, mobile apps, APIs, databases, microservices, etc.) within the system.
- Shows how **containers communicate**.
- Helps in deployment and infrastructure planning.

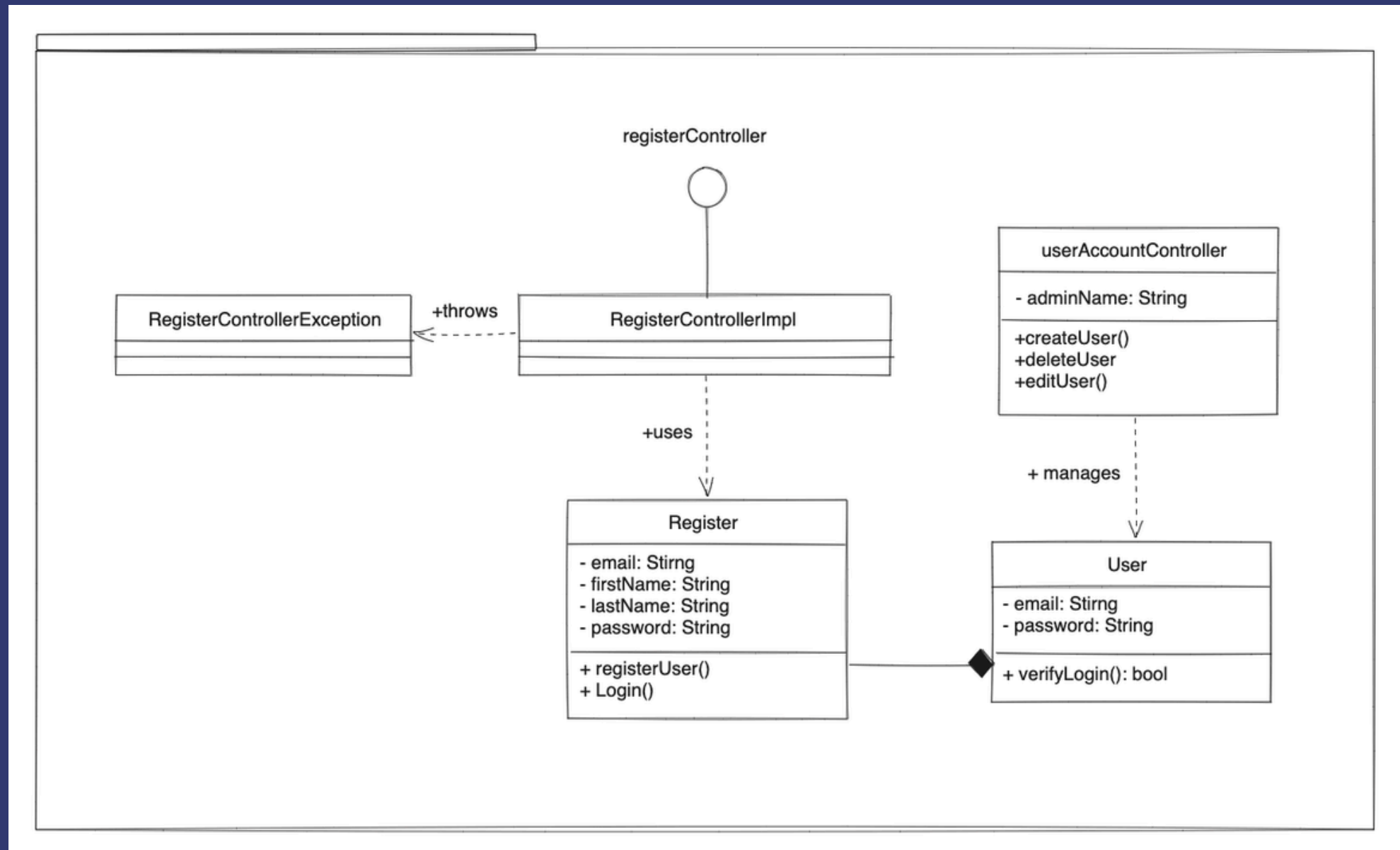
Component

- Zooms into **one container** (like a backend service) and shows its internal components/modules, e.g. service classes, controllers or data access layers.
- Useful for devs to understand internal design and division of responsibilities.

**level three:
component diagram**

Offers a detailed view of the source code structure (e.g. classes and interfaces) within a component.

Different styles, but more or less the same as the UML diagrams we did just before ass1.



level four:
code(class) diagram



You're designing a university Course Management System used by students and instructors.

The system allows:

- Students to **view and submit assignments**, receive **grades** and participate in **forums**
- Instructors to **create and grade assignments**, post **announcements** and **manage students**
- The system has a **web frontend, backend, database** and integrates with other university systems like **authentication and email**.

moodle example



diagram demo!

Context & Container Diagrams



discuss.



Currently, all functionality of the Course Management System is implemented within a single monolithic backend.

What might be the benefits of splitting this monolith into smaller, independent services, and how could we divide the system's functionality to achieve better modularity, cohesion, and scalability?





Monolithic Backend

- All functionality (user management, course handling, grading, forums, etc.) exists in one large codebase and is deployed together.
- Simpler initially, but becomes harder to maintain and scale as the system grows.

Independent Services

- Better modularity and cohesion (teams can work on their specific services and not have to touch other parts of the backend). Codebases are then smaller, easier to understand and test.
- Decoupling backend allows for independent deployability and fault isolation

monolithic vs. independent

Could divide the CMS into major business domains or system responsibilities:

- **Assessment Service:** Manages assignment creation, submissions and deadlines; handles grading logic; releases notifications for instructors
- **Forum Service:** Manages discussion threads, posts and announcements
- **Course Service:** Handles course creation, organising course info (e.g. weeks, modules, lecture topics, tutorials)



example architecture

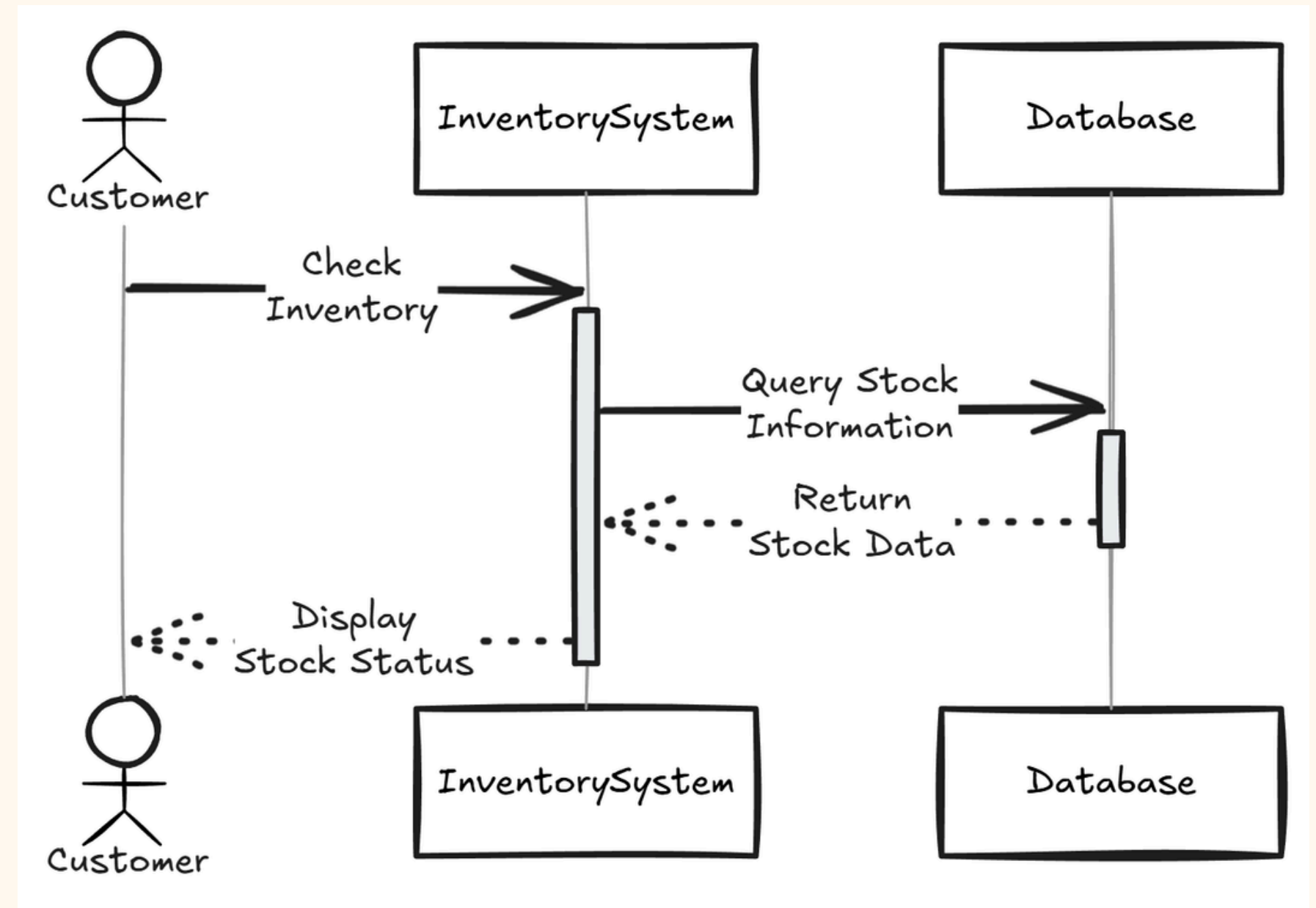


sequence diagrams

Wondrous



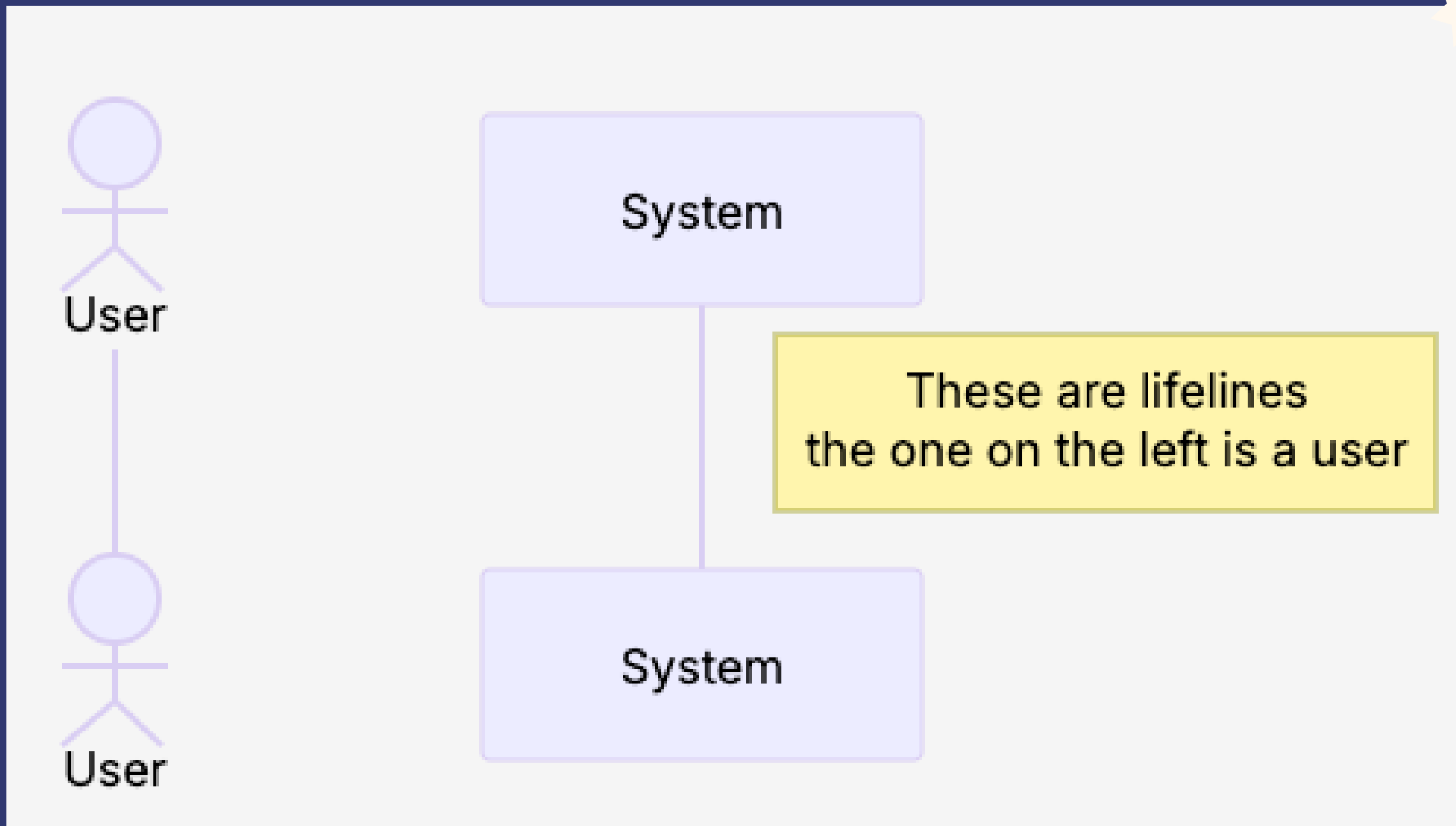
- An interaction diagram showing the **temporal order of interactions** between objects or components to achieve a **specific functionality or use case**
- Shows **how operations are carried out through message exchanges**
- Helpful for **visualising the flow** of control and messages over time.



sequence diagrams ★

lifelines

- Vertical lines that represent the timeline of a specific entity.
- Top of the lifeline is the name of the entity (usually in a rectangle). Each entity has exactly one lifeline.
- To distinguish users from other entities in the system, the name of the user is written below a stick figure in place of the rectangle.

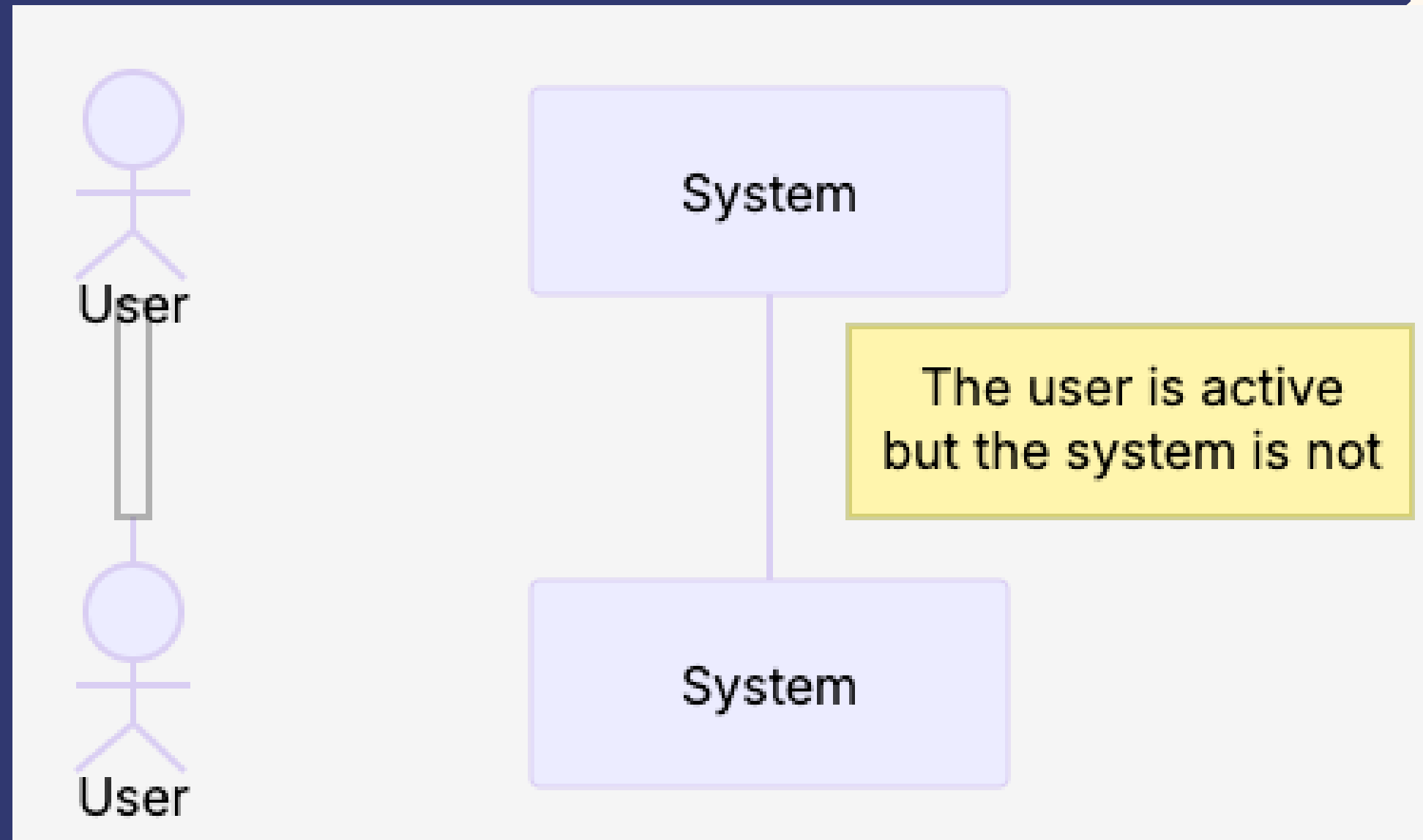


sequenceDiagram

actor User

participant System

Note right of System: These are lifelines
 the one on the left is a user



activation boxes

- AKA execution specifications
- Thin rectangular boxes on lifelines that denote when a particular entity is active.
- An entity is activated when sending/receiving a message (remains active while executing the action requested in the message) and while waiting for a response

sequenceDiagram

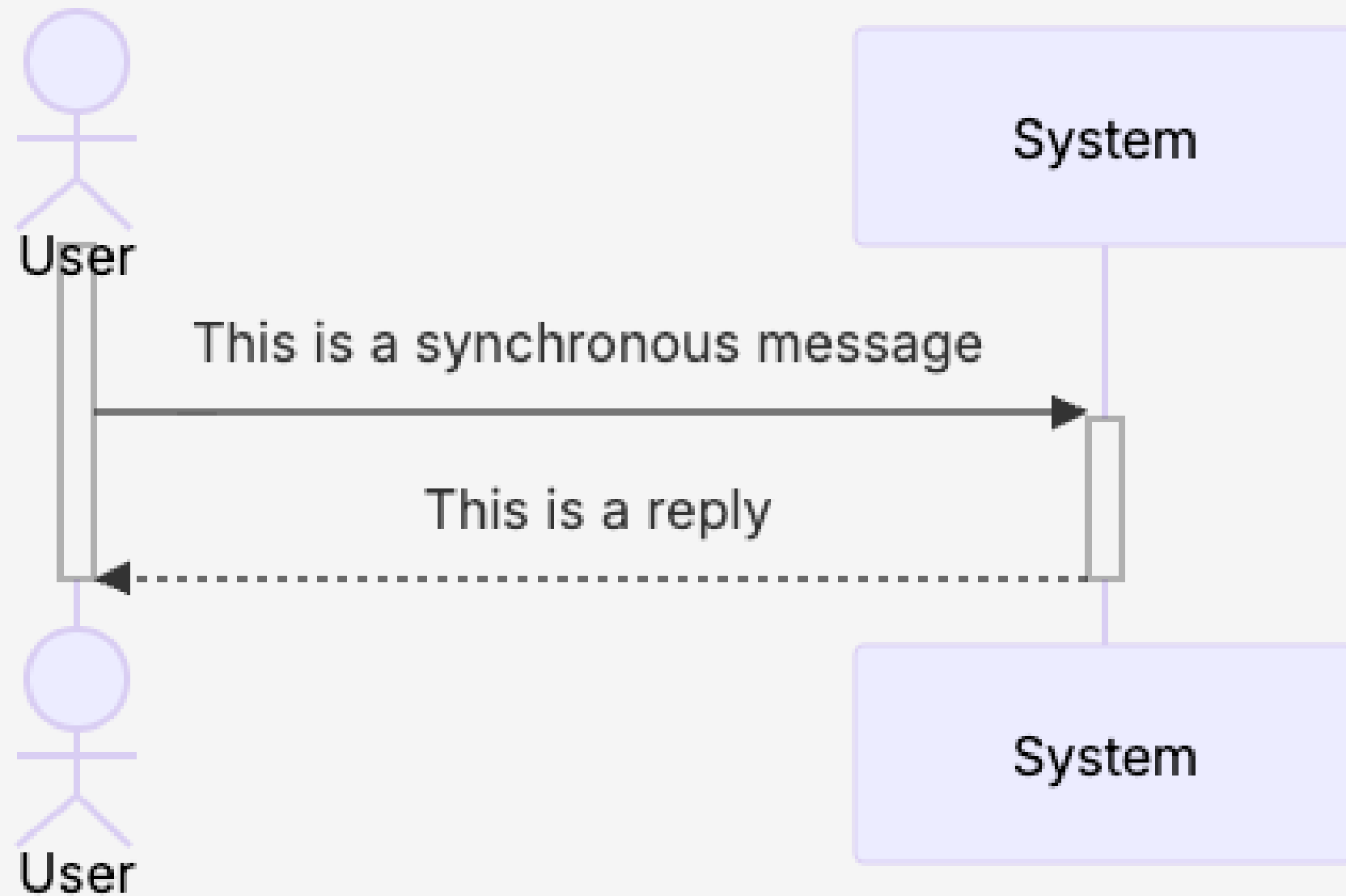
actor User

participant System

activate User

Note right of System: The user is active
 but the system is not

deactivate User



synchronous messages & replies

- **Messages:** when one entity is making a request from another entity and expects a response in return. The sender waits for the receiver to complete the operation and return a response.
- **Replies:** sent in response to receiving a synchronous message. They either contain the data the sender requested or a confirmation that the requested action has been taken. All synchronous messages should have a corresponding reply.

sequenceDiagram

actor User

participant System

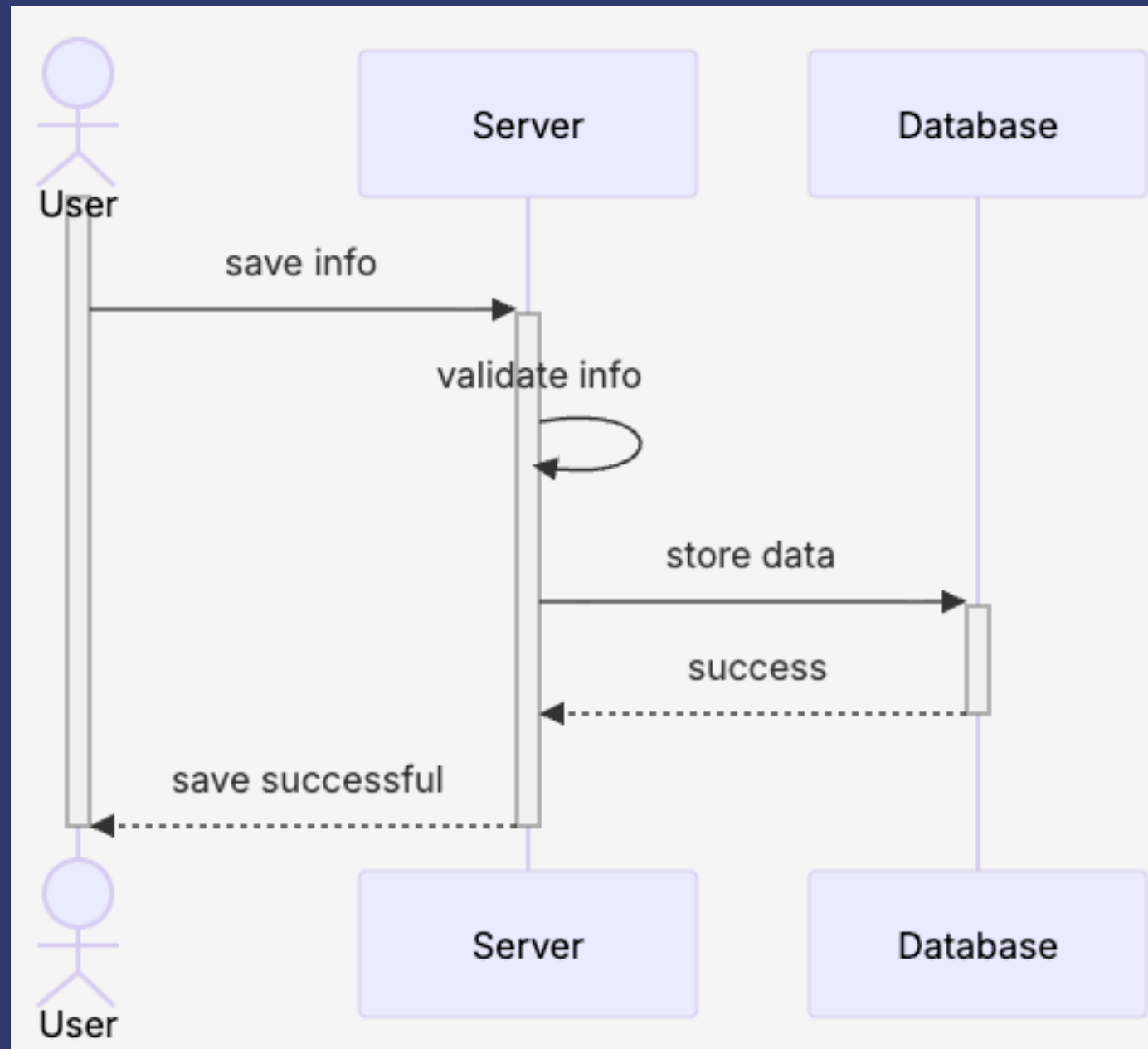
activate User

User->>+System: This is a synchronous message

System-->>-User: This is a reply

deactivate User

self loops



sequenceDiagram

actor User

participant Server

participant Database

activate User

User->>+Server: save info

Server->>Server: validate info

Server->>+Database: store data

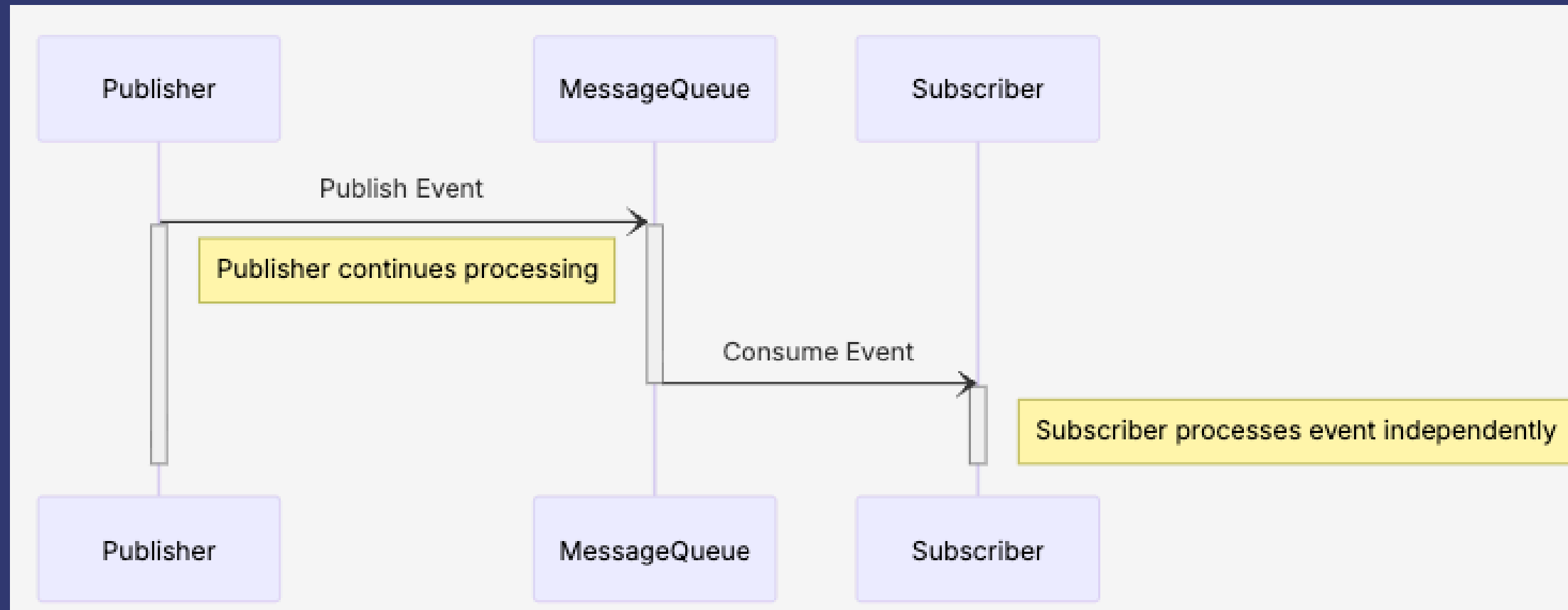
Database-->>-Server: success

Server-->>-User: save successful

deactivate User

- A entity can send a message to itself. This is called a self loop.
- Used when an entity needs to do **additional processing before continuing**.
- Often used to show **data validation and calculation tasks** that would be performed internally, but should be noted in the sequence of events.
- Does not need a reply as the reply is implied

async messages



sequenceDiagram

... [insert participant setup here]

Publisher->>MessageQueue: Publish Event

activate Publisher

note right of Publisher: Publisher continues processing

MessageQueue->>Subscriber: Consume Event

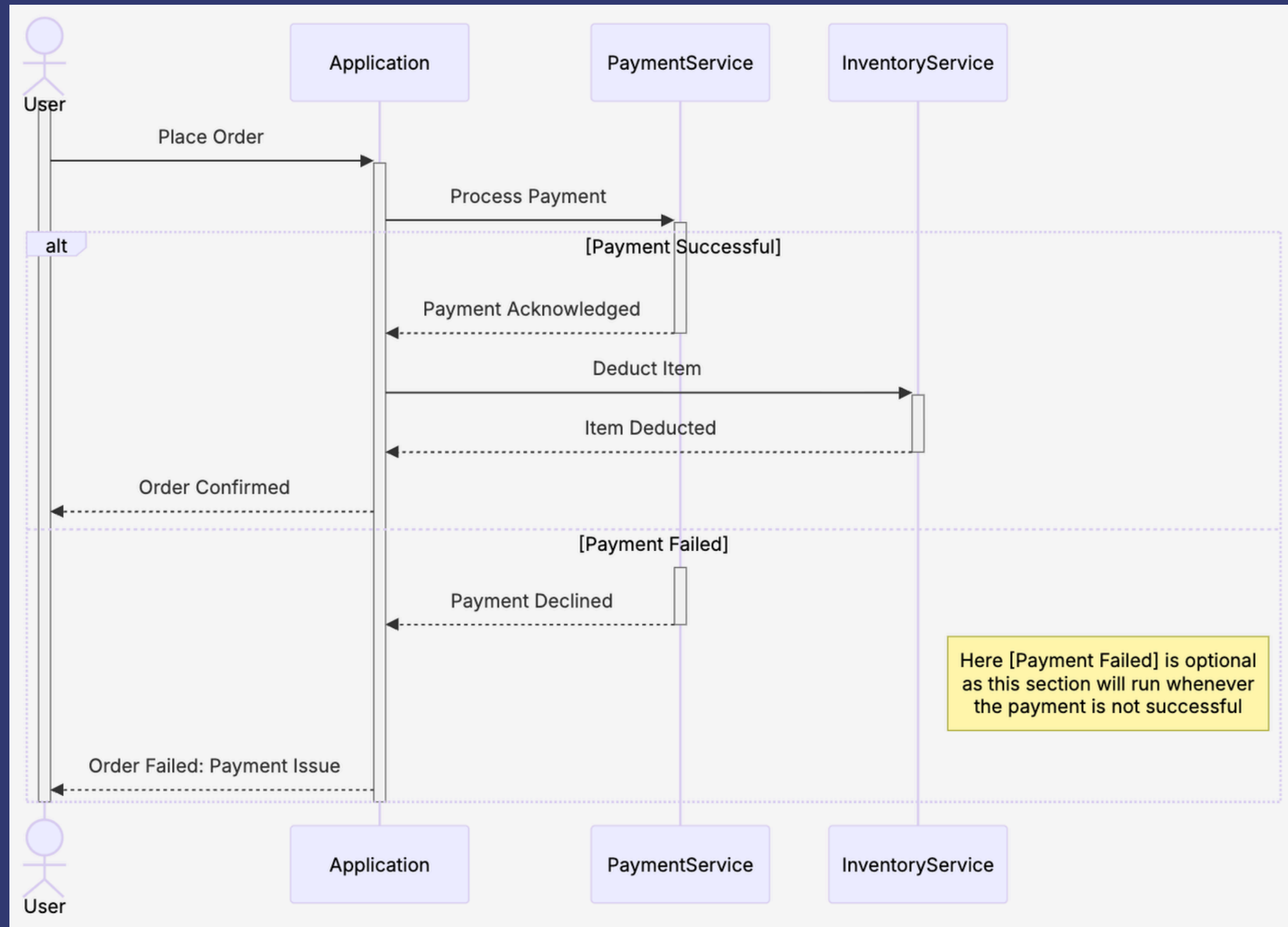
activate Subscriber

note right of Subscriber: Subscriber processes event independently

... [insert deactivations here]

- Used when the sender does not wait for the receiver to complete the operation
- Sends the message and continues its own execution immediately
- Asynchronous
messages don't require a reply, but one can be sent if it makes sense to do so.
- Solid line, open arrowhead

alternatives (alt)



- Used to model conditional statements with 2+ possible options (an if...else statement)
- Guard condition at the top of the fragment box.
- Each alternative needs its own guard except for the last, as it is treated like an else (but a guard can be included to improve readability) .

alternatives (alt)

```
sequenceDiagram
```

```
...
```

```
Application->>+PaymentService: Process Payment
```

```
alt Payment Successful
```

```
    PaymentService-->>-Application: Payment
```

```
Acknowledged
```

```
    Application->>+InventoryService: Deduct Item
```

```
    InventoryService-->>-Application: Item Deducted
```

```
    Application-->>User: Order Confirmed
```

```
else Payment Failed
```

```
    activate PaymentService
```

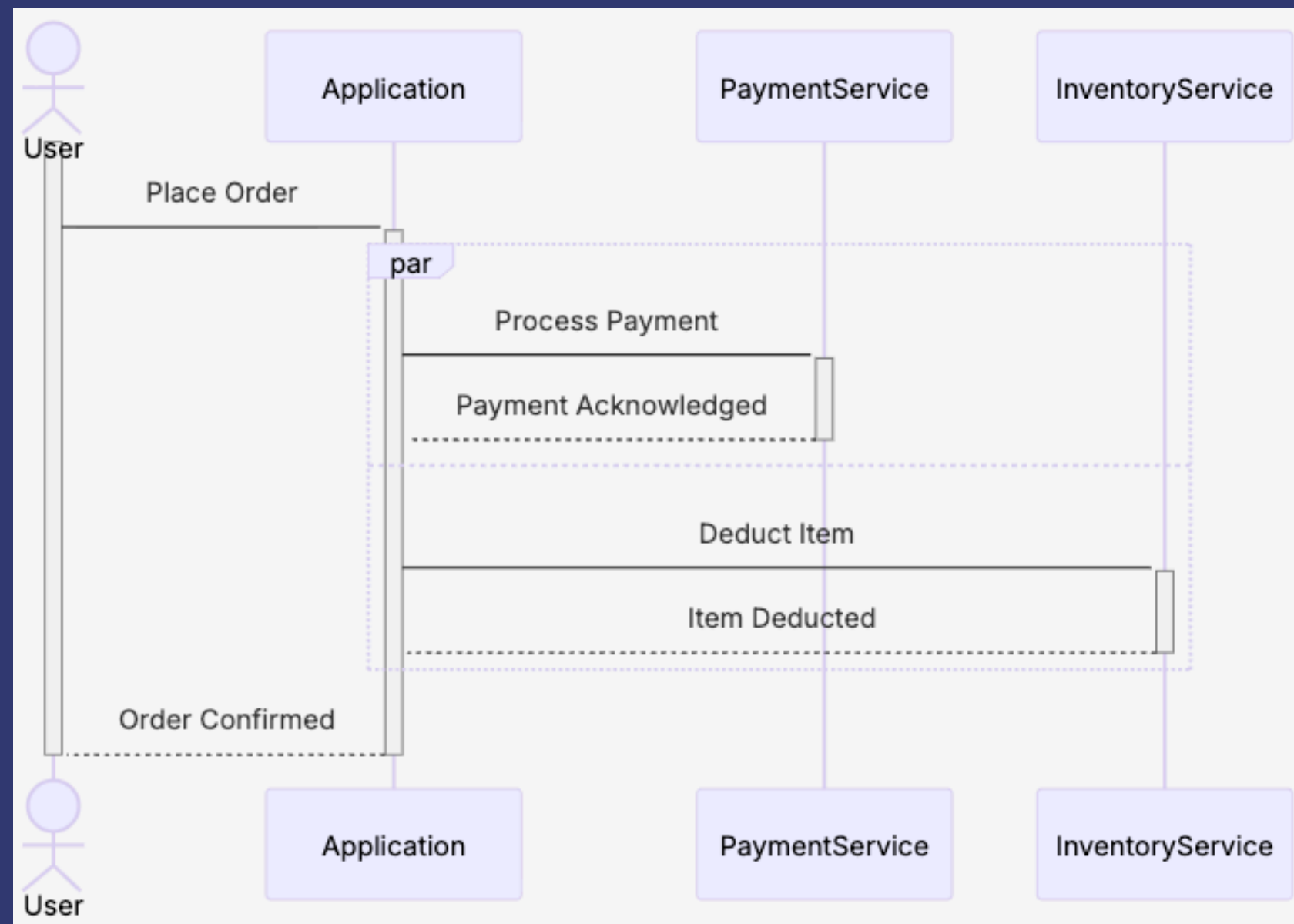
```
    PaymentService-->>-Application: Payment Declined
```

```
    Application-->>User: Order Failed: Payment Issue
```

```
end
```

```
...
```

- Used to model conditional statements with 2+ possible options (an if...else statement)
- Guard condition at the top of the fragment box.
- Each alternative needs its own guard except for the last, as it is treated like an else (but a guard can be included to improve readability) .



- Used to convey concurrent actions.
- Contains 2 or more sections that are run concurrently.
- Do not need a guard as parallel fragments are not optional.

sequenceDiagram

...

activate User

User->>+Application: Place Order

par

Application->>+PaymentService: Process Payment

PaymentService-->>-Application: Payment Acknowledged

and

Application->>+InventoryService: Deduct Item

InventoryService-->>-Application: Item Deducted

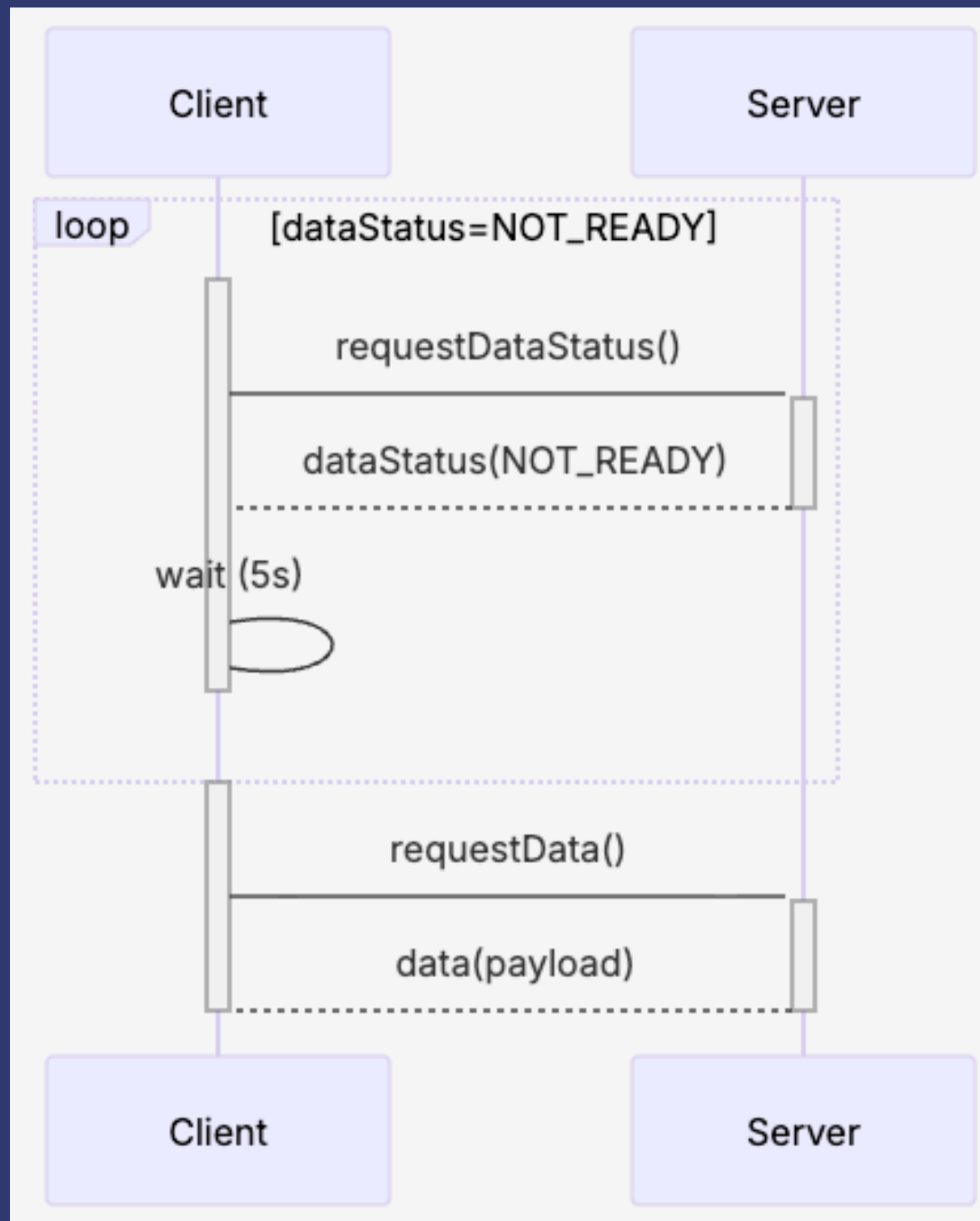
end

Application-->>User: Order Confirmed

parallel (par)

loop

- Loop fragments are used to convey repeated actions.
- They contain one section that is repeated while the guard condition is true.



sequenceDiagram

...

loop `dataStatus=NOT_READY`

activate Client

Client->>+Server: `requestDataStatus()`

Server-->>-Client: `dataStatus(NOT_READY)`

Client->>Client: `wait (5s)`

deactivate Client

end

activate Client

Client->>+Server: `requestData()`

Server-->>-Client: `data(payload)`

deactivate Client

break

- A termination of the current flow. Guard in the top center of the box represents the break condition.
- The actions within the break are then performed instead of the remainder of the enclosing fragment/sequence diagram

sequenceDiagram

...

activate Client

Client->>+Server: Request Data batch

loop For each record

 Server->>+Database: Fetch Record

 Database-->>-Server: Return Record

 break error fetching record

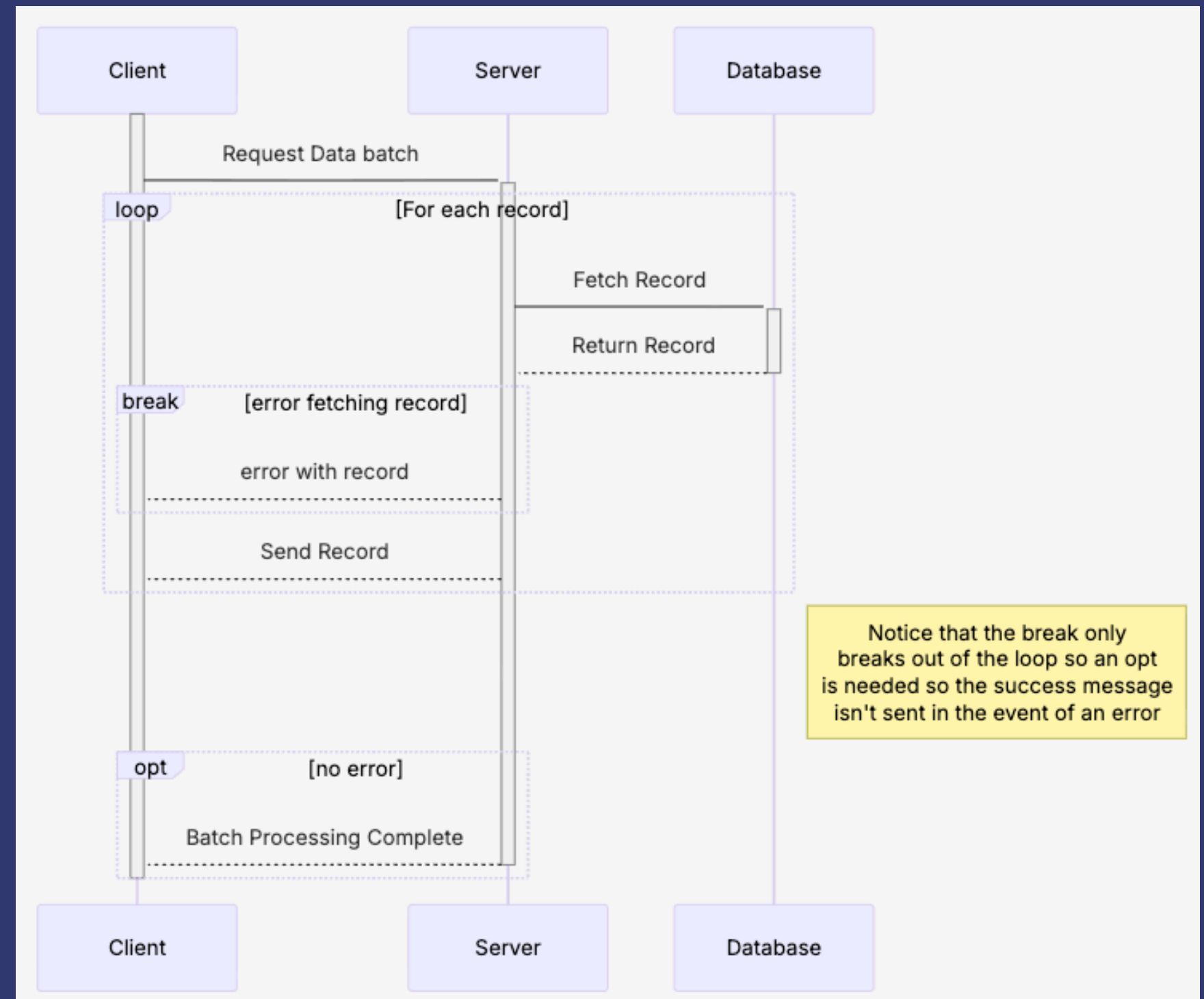
 Server-->>Client: error with record

 end

 Server-->>Client: Send Record

end

...



✦ ✦ ✦

moodle example pt. two

You are part of the development team designing the Course Management System. Create a **Sequence Diagram** illustrating the flow when an instructor making an assessment with a deadline.



code demo

Mermaid Sequence Diagram





lab time!

SitarME306